SQL Injection Finding Report

Enterprise DevSecOps Security Lab

Finding ID: APP-01

SQL Injection in Flask /user Endpoint

HIGH RISK

Document Type	Application Security Finding Report
Author	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Target Component	Vulnerable Flask application - /user endpoint
Assessment Mode	Private VM lab / controlled local testing
Classification	Portfolio security assessment documentation

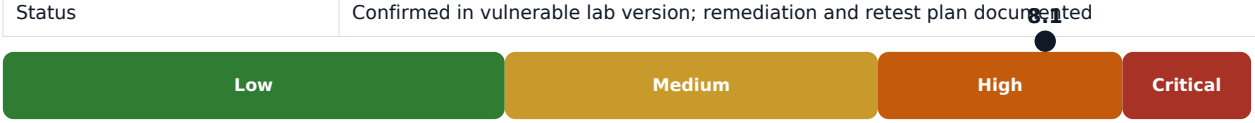
This report documents a SQL Injection vulnerability discovered in the deliberately vulnerable Flask application used in the Enterprise DevSecOps lab. It includes risk scoring, exploit validation, root cause analysis, evidence, remediation guidance, OWASP mapping, and retest criteria.

1. Executive Summary

The assessment identified a SQL Injection vulnerability in the Flask application route /user. The route accepts a user-controlled id parameter and places it directly into a SQL statement using Python f-string interpolation. A crafted payload changed the query logic and returned multiple rows from the users table instead of a single expected record.

The issue is intentionally present for private lab testing; however, the pattern represents a high-risk production weakness. In a real application, similar query construction can allow unauthorized data access, authentication bypass, data modification, data deletion, and attacker-driven database enumeration. The primary remediation is to separate SQL code from user-supplied data by using parameterized queries or a safe ORM query interface.

Field	Value
Finding ID	APP-01
Finding Name	SQL Injection in Flask /user Endpoint
Affected Route	/user?id=<user supplied value>
Severity	High
CVSS v3.1 Style Score	8.1 - High (environment-adjusted lab score)
CWE Mapping	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command
OWASP Top 10 Mapping	A03:2021 - Injection
OWASP ASVS Mapping	ASVS 5.0.0 V1.2.4 - database queries should use parameterized queries, ORMs, or equivalent protection
Status	Confirmed in vulnerable lab version; remediation and retest plan documented



Risk rating note: The score is calibrated for the lab context where the database is small and opened read-only at runtime. In a production environment with sensitive records, authenticated workflows, write-capable database permissions, or chained access to additional tables, this finding could be rated higher.

2. Scope and Affected Asset

The finding applies to the intentionally vulnerable Flask application included in the Enterprise DevSecOps project. Testing was performed inside a controlled private VM lab environment using local requests against the application endpoint.

Asset	Details
Application	Python Flask vulnerable demo application
File	src/app/app.py
Function	get_user()
HTTP Method	GET
Route	/user
Parameter	id
Database	SQLite users.db
Table	users
Test Type	Manual payload validation + source code review + SAST correlation

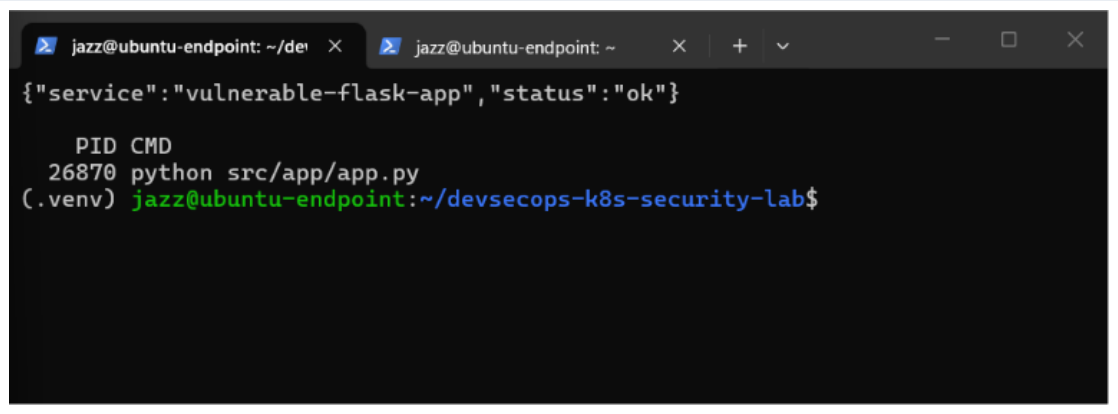


Figure 1 - Vulnerable Flask application running locally in the private lab.

3. Technical Vulnerability Details

The root cause is direct construction of a SQL query with untrusted input. The endpoint reads the request parameter id, stores it in user_id, and embeds that value into a SQL string before execution. Because the database receives the final string as executable SQL, an attacker can use quotes and Boolean logic to alter the meaning of the WHERE clause.

Vulnerable code pattern

```
@app.route("/user")
def get_user():
    user_id = request.args.get("id", "1")

    conn = sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True)
    cursor = conn.cursor()

    query = f"SELECT id, username, role FROM users WHERE id = '{user_id}'"

    result = cursor.execute(query).fetchall()
    conn.close()

    return {
        "query_used": query,
        "result": result
    }
```

The query construction causes user input to be treated as SQL syntax rather than as a safe value. The following comparison shows the intended logic and the attacker-modified logic.

Case	Request Input	Effective SQL Logic	Result
Normal request	id=1	WHERE id = '1'	Returns only the user with id 1
Injected request	id=1' OR '1'='1	WHERE id = '1' OR '1'='1'	Returns all rows because the OR condition is always true

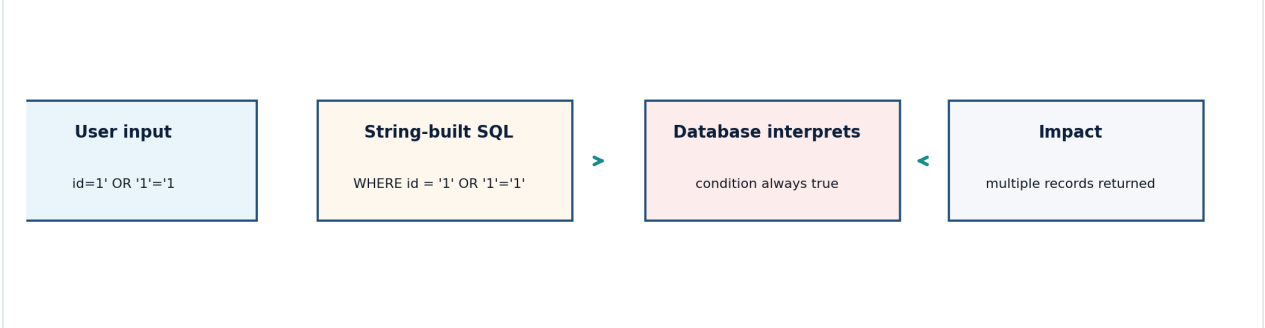


Figure 2 - SQL injection logic flow for the vulnerable /user endpoint.

4. Proof of Concept

The proof of concept was executed only against the local lab application. The goal was to validate whether the endpoint expands the result set when Boolean SQL syntax is inserted through the id parameter.

Baseline request

```
curl "http://localhost:5000/user?id=1"
```

Expected baseline behavior: the application should return one record matching id 1.

Injected request

```
curl "http://localhost:5000/user?id=1'%20OR%20'1'%3D'1"
```

Observed vulnerable behavior: the response contained multiple records from the users table, demonstrating that the attacker-controlled input was interpreted as SQL logic.

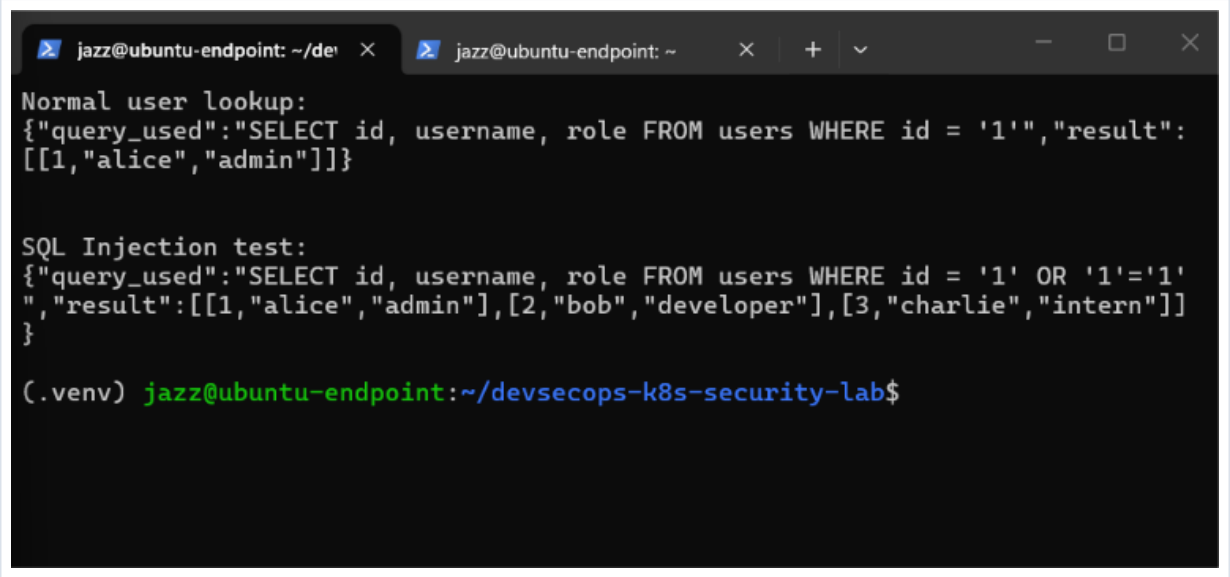


Figure 3 - Manual SQL injection proof showing expanded result set from the /user endpoint.

5. Root Cause Analysis

The finding is caused by unsafe query construction. The application uses a SQL interpreter, and the interpreter receives a single string containing both trusted query syntax and untrusted request data. Because there is no parameter binding boundary, the database cannot distinguish the intended value from injected SQL syntax.

Root Cause Area	Observation	Security Failure
Input handling	The id parameter is accepted as a raw string.	No positive validation or type enforcement before database use.
Query construction	The parameter is interpolated into a SQL statement.	Data and SQL syntax are mixed into the same execution string.
Database access	SQLite query is executed with cursor.execute(query).	The final query structure can be modified by the attacker.
Testing coverage	The lab demonstrates the issue manually and with security scanning.	Secure regression tests should be added after remediation.

Exploitability Factors

- The vulnerable parameter is reachable through a simple GET request.
- No authentication or authorization is required in the lab route.
- The payload uses common SQL syntax and does not require advanced tooling.
- The application returns the query used and the database result, increasing feedback for the tester.
- The proof-of-concept confirms result expansion rather than a purely theoretical code issue.

6. Security Impact

In this private lab, the database contains demonstration users only. In a production application, the same weakness could expose account data, authorization attributes, sensitive business records, or internal application state. SQL injection is especially important because it can directly affect confidentiality and integrity at the data layer.

Impact Area	Lab Observation	Potential Production Impact
Confidentiality	Multiple user rows were returned from the users table.	Unauthorized extraction of user, business, or operational data.
Integrity	The tested SQLite connection is read-only.	If write-capable database access exists, attackers may alter or delete records.
Authentication / Authorization	The demo route has no access control.	SQLi could bypass login checks or role filters if similar code appears in auth flows.
Detection	The app returns query details in the response.	Verbose error/query exposure can accelerate exploitation and enumeration.
Chaining	The app also contains other intentional vulnerabilities.	SQLi may be chained with command injection, weak dependencies, or misconfiguration.

7. Standards and Control Mapping

The finding maps directly to SQL injection weakness classifications and secure development requirements. The key control principle is to ensure untrusted input is never interpreted as SQL code.

Framework	Mapping	Reason
OWASP Top 10	A03:2021 - Injection	SQL injection is a common injection class where attacker data can influence queries or commands.
OWASP ASVS	v5.0.0 V1.2.4	Requires data selection or database queries to use parameterized queries, ORMs, entity frameworks, or equivalent protection from database injection attacks.
CWE	CWE-89	Improper neutralization of SQL special elements before command execution.
OWASP Cheat Sheet	SQL Injection Prevention / Query Parameterization	Primary recommendation is to avoid dynamic string-built queries and use parameterized statements.

8. SAST and Evidence Correlation

The finding is supported by both manual testing and source-code review. Bandit also identified SQL injection-related risk through static analysis. In professional reporting, this correlation is useful because it shows that the issue is reproducible through runtime testing and traceable to a specific insecure code pattern.

Figure 4 - Bandit SAST evidence highlighting SQL injection risk and related insecure patterns.

Evidence correlation matrix

Evidence Source	What It Proves	Value to Remediation
Manual curl payload	The application returns expanded rows with Boolean SQL injection.	Confirms the issue is exploitable in the lab runtime.
Source code review	The route builds SQL with f-string interpolation.	Identifies the exact root cause to fix.
Bandit SAST	Static tool flags SQL injection construction pattern.	Can be used as a CI/CD guardrail after remediation.
OWASP/CWE mapping	Confirms recognized weakness classification.	Supports risk acceptance, remediation prioritization, and reporting consistency.

9. Remediation Guidance

The recommended remediation is to use a parameterized query and treat id as data only. Input validation should be used to enforce expected format and type, but validation alone should not replace parameterized SQL. The database call should bind the value separately from the SQL statement.

Recommended secure code pattern

```
from flask import abort

@app.route("/user")
def get_user():
    raw_user_id = request.args.get("id", "1")

    try:
        user_id = int(raw_user_id)
    except ValueError:
        abort(400, description="Invalid user id")

    conn = sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True)
    cursor = conn.cursor()

    cursor.execute(
        "SELECT id, username, role FROM users WHERE id = ?",
        (user_id,)
    )

    result = cursor.fetchall()
    conn.close()

    return {"result": result}
```

Why this fixes the issue

- The SQL statement structure remains constant and cannot be modified by the id value.
- The placeholder (?) tells SQLite to bind the input as a value rather than SQL syntax.
- The integer conversion rejects payloads containing quotes, spaces, Boolean operators, or SQL fragments.
- Removing query_used from the response reduces attacker feedback and prevents query disclosure.
- The pattern is easy to enforce with unit tests and SAST/SCA security gates.

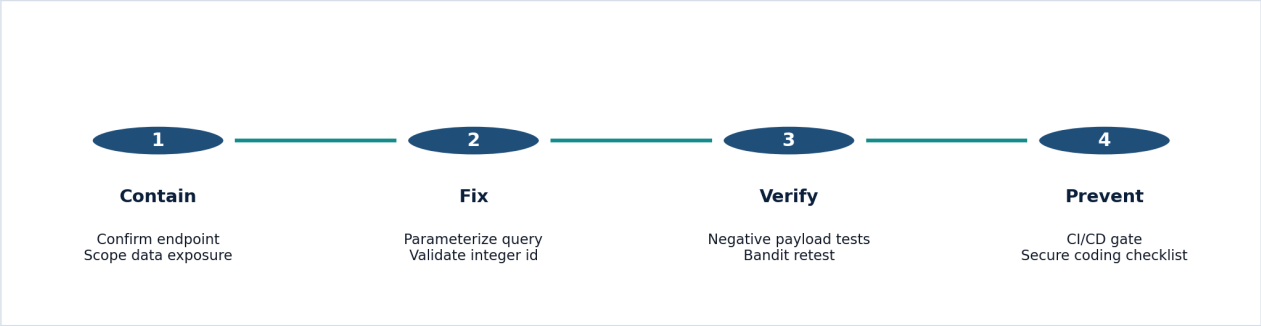


Figure 5 - Recommended remediation and validation sequence.

10. Retest Plan and Acceptance Criteria

After the fix is applied, the endpoint should be retested with both valid and malicious input. The fix is not considered complete until normal functionality is preserved, injection payloads are rejected or treated as inert data, and automated security tooling no longer reports the vulnerable query construction pattern.

Test ID	Test Case	Expected Result	Pass Criteria
SQLI-RT-01	GET /user?id=1	Returns only the expected user record.	One row for id 1; no query disclosure.
SQLI-RT-02	GET /user?id=1' OR '1'='1	Rejected as invalid id or returns no expanded result.	No multi-row expansion.
SQLI-RT-03	GET /user?id=abc	Rejected with 400 Bad Request.	No stack trace or SQL error disclosure.
SQLI-RT-04	GET /user?id=1;DROP TABLE users	Rejected as invalid id.	No database error, no query execution beyond safe SELECT.
SQLI-RT-05	Run Bandit against src/app	No B608 finding for /user route.	SAST report confirms the vulnerable pattern is removed.
SQLI-RT-06	Run application unit tests	Valid and invalid id tests pass.	Regression tests added to CI/CD pipeline.

Recommended security unit tests

```
def test_user_endpoint_rejects_sql_injection(client):
    response = client.get("/user?id=1' OR '1'='1")
    assert response.status_code in (400, 422)


def test_user_endpoint_accepts_integer_id(client):
    response = client.get("/user?id=1")
    assert response.status_code == 200
    assert len(response.json["result"]) == 1
```

11. DevSecOps Integration Recommendations

The finding should be remediated in code and reinforced through the pipeline. The goal is not only to fix one route, but to prevent recurrence of string-built queries across the application.

Control Layer	Recommended Control	Implementation Notes
Developer workflow	Secure coding checklist for database access	Require parameterized queries or ORM methods for every database interaction.
Code review	SQL construction review rule	Reviewers should flag string concatenation/interpolation inside database calls.
SAST	Bandit job in GitHub Actions	Keep Bandit running on every push and pull request; fail or gate on high-confidence SQLi findings.
Testing	Negative AppSec tests	Add SQLi payload regression tests for all routes that use request parameters.
Dependency management	pip-audit and SBOM generation	Ensure database libraries and frameworks are inventoried and patched.
Runtime monitoring	Application logs + Falco correlation	Log rejected payloads safely and correlate with runtime detections where applicable.

12. Closure Checklist

Closure Item	Required Evidence
Vulnerable code removed	Code diff showing parameterized query and validation added.
Payload retested	Screenshots or test output proving Boolean SQLi no longer expands results.
SAST clean or accepted	Bandit report showing no SQLi finding for the route, or documented false positive if applicable.
Regression tests added	Unit/integration tests covering normal, malformed, and injection payload inputs.
Response hardened	No query_used or raw SQL error details returned to the client.
Pipeline updated	CI/CD job retains Bandit and negative test execution on pull requests.

13. References

The following public references were used to align the finding classification and remediation guidance:

- OWASP Top 10: A03:2021 - Injection, https://owasp.org/Top10/2021/A03_2021-Injection/
- OWASP SQL Injection Prevention Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Query Parameterization Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html
- OWASP ASVS 5.0.0 V1 Encoding and Sanitization, https://github.com/OWASP/ASVS/blob/master/5.0/en/0x10-V1-Encoding-and-Sanitization.md
- MITRE CWE-89, https://cwe.mitre.org/data/definitions/89.html